

Lab Assignment # 02

Course Title : **AI Assistant Coding**
Name of Student : **M. Sai Pallavi**
Enrollment No. : **2303A54063**
Batch No. : **48**

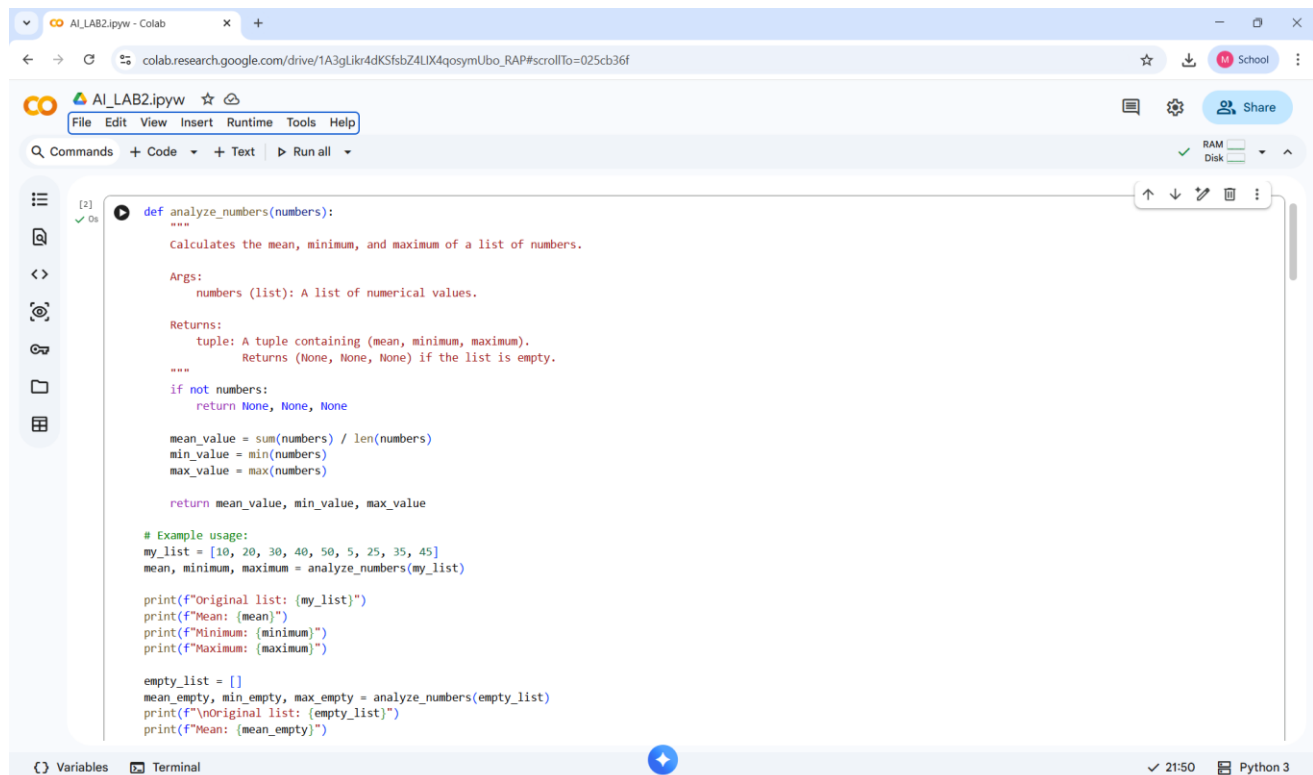
Lab 2: Exploring Additional AI Coding Tools beyond Copilot – Gemini (Colab) and Cursor AI

Task 1: Statistical Summary for Survey Data

❖ *Scenario: You are a data analyst intern working with survey responses stored as numerical lists.*

• **Prompt used:** Write a Python function that takes a list of numbers and returns the mean, minimum, and maximum values.

• Screenshot of Generated Code:



```
[2] ✓ Os
def analyze_numbers(numbers):
    """
    Calculates the mean, minimum, and maximum of a list of numbers.

    Args:
        numbers (list): A list of numerical values.

    Returns:
        tuple: A tuple containing (mean, minimum, maximum).
        Returns (None, None, None) if the list is empty.
    """
    if not numbers:
        return None, None, None

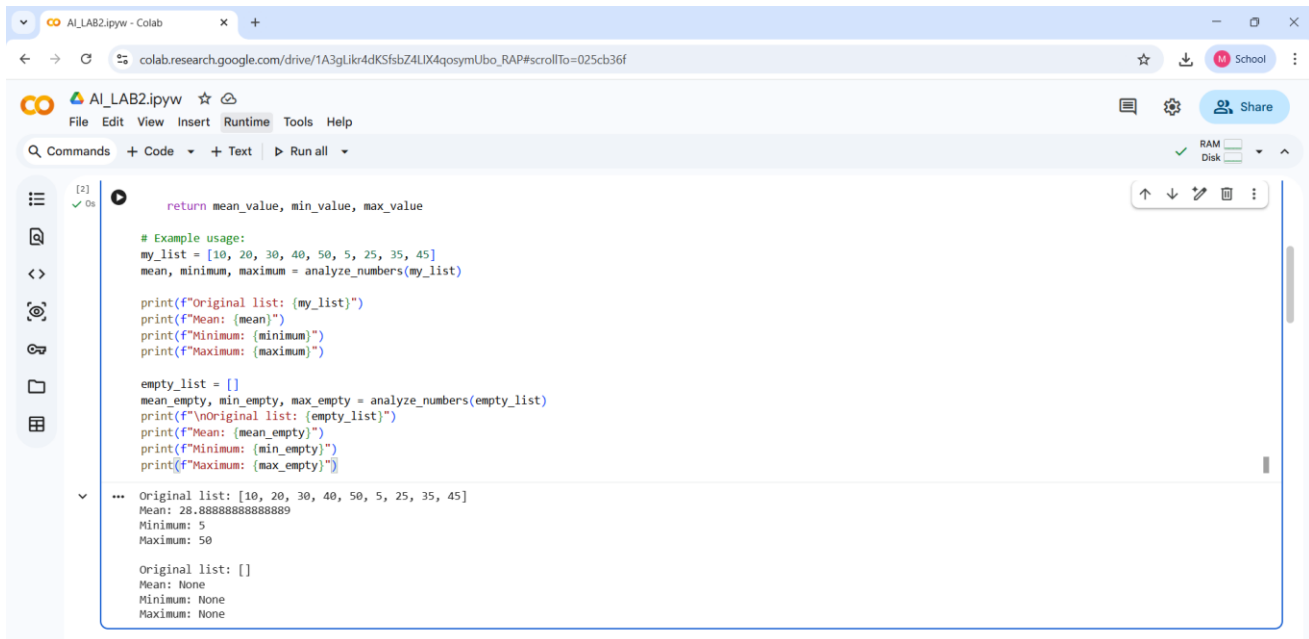
    mean_value = sum(numbers) / len(numbers)
    min_value = min(numbers)
    max_value = max(numbers)

    return mean_value, min_value, max_value

# Example usage:
my_list = [10, 20, 30, 40, 50, 5, 25, 35, 45]
mean, minimum, maximum = analyze_numbers(my_list)

print(f"Original list: {my_list}")
print(f"Mean: {mean}")
print(f"Minimum: {minimum}")
print(f"Maximum: {maximum}")

empty_list = []
mean_empty, min_empty, max_empty = analyze_numbers(empty_list)
print(f"Original list: {empty_list}")
print(f"Mean: {mean_empty}")
```



```
def analyze_numbers(numbers):  
    mean_value = sum(numbers) / len(numbers)  
    minimum = min(numbers)  
    maximum = max(numbers)  
    return mean_value, minimum, maximum  
  
# Example usage:  
my_list = [10, 20, 30, 40, 50, 5, 25, 35, 45]  
mean, minimum, maximum = analyze_numbers(my_list)  
  
print(f"Original list: {my_list}")  
print(f"Mean: {mean}")  
print(f"Minimum: {minimum}")  
print(f"Maximum: {maximum}")  
  
empty_list = []  
mean_empty, min_empty, max_empty = analyze_numbers(empty_list)  
print(f"Original list: {empty_list}")  
print(f"Mean: {mean_empty}")  
print(f"Minimum: {min_empty}")  
print(f"Maximum: {max_empty}")
```

Original list: [10, 20, 30, 40, 50, 5, 25, 35, 45]
Mean: 28.88888888888889
Minimum: 5
Maximum: 50

Original list: []
Mean: None
Minimum: None
Maximum: None

Short Explanation of Logic:

The mean represents the average of a set of numbers. It is obtained by adding all the values in the list and dividing the sum by the total number of elements. The minimum value is the smallest number present in the list, and the maximum value is the largest number present in the list. These values are calculated using Python's built-in min() and max() functions. The function accepts a list of numerical values as input, computes the mean, minimum.

Task 2: Armstrong Number – AI Comparison

❖ *Scenario: You are evaluating AI tools for numeric validation logic.*

- **Prompt used (Gemini):** "Write a Python program to check whether a given number is an Armstrong number."
- **Prompt used (Copilot):** "Write a Python program to check whether a given number is an Armstrong number."

• Screenshot of Generated Code:

```
AI_LAB2.ipynw
colab.research.google.com/drive/1A3gLikr4dKSfsbZ4LIX4qosymUbo_RAP#scrollTo=025cb36f

AI_LAB2.ipynw
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
RAM Disk

[3] ✓ 0s
num_digits = len(num_str)
sum_of_powers = 0

for digit_char in num_str:
    digit = int(digit_char)
    sum_of_powers += digit ** num_digits

return sum_of_powers == number

# Example usage:
print(f"Is 153 an Armstrong number? {is_armstrong_number(153)}") # Expected: True (1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153)
print(f"Is 370 an Armstrong number? {is_armstrong_number(370)}") # Expected: True (3^3 + 7^3 + 0^3 = 27 + 343 + 0 = 370)
print(f"Is 9474 an Armstrong number? {is_armstrong_number(9474)}") # Expected: True (9^4 + 4^4 + 7^4 + 4^4 = 6561 + 256 + 2401 + 256 = 9474)
print(f"Is 10 an Armstrong number? {is_armstrong_number(10)}") # Expected: False
print(f"Is 1 an Armstrong number? {is_armstrong_number(1)}") # Expected: True (1^1 = 1)

Is 153 an Armstrong number? True
Is 370 an Armstrong number? True
Is 9474 an Armstrong number? True
Is 10 an Armstrong number? False
Is 1 an Armstrong number? True
```

```
AI_LAB2.ipynw
colab.research.google.com/drive/1A3gLikr4dKSfsbZ4LIX4qosymUbo_RAP#scrollTo=025cb36f

AI_LAB2.ipynw
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
RAM Disk

Git Hub Co-pilot code

[6] ✓ 0s
def is_armstrong(number: int) -> bool:
    """
    Check if a given number is an Armstrong number.

    An Armstrong number of 'n' digits is a number such that
    the sum of its digits raised to the power 'n' equals the number itself.
    """
    # Convert number to string to easily iterate over digits
    digits = str(number)
    num_digits = len(digits)

    # Compute sum of each digit raised to the power of num_digits
    total = sum(int(digit) ** num_digits for digit in digits)

    return total == number

# Example usage
if __name__ == "__main__":
    test_numbers = [153, 9474, 123, 407]
    for num in test_numbers:
        if is_armstrong(num):
            print(f"{num} is an Armstrong number.")
        else:
            print(f"{num} is NOT an Armstrong number.")

153 is an Armstrong number.
9474 is an Armstrong number.
123 is NOT an Armstrong number.
407 is an Armstrong number.
```

```
AI_LAB2.ipynw
colab.research.google.com/drive/1A3gLikr4dKSfsbZ4LIX4qosymUbo_RAP#scrollTo=025cb36f

AI_LAB2.ipynw
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
RAM Disk

[3] ✓ 0s
def is_armstrong_number(number):
    """
    Checks if a number is an Armstrong number.

    An Armstrong number is a number that is the sum of its own digits
    each raised to the power of the number of digits.

    Args:
        number (int): The integer to check.

    Returns:
        bool: True if the number is an Armstrong number, False otherwise.
    """
    if not isinstance(number, int) or number < 0:
        raise ValueError("Input must be a non-negative integer.")

    # Convert number to string to get its digits and count them
    num_str = str(number)
    num_digits = len(num_str)
    sum_of_powers = 0

    for digit_char in num_str:
        digit = int(digit_char)
        sum_of_powers += digit ** num_digits

    return sum_of_powers == number
```

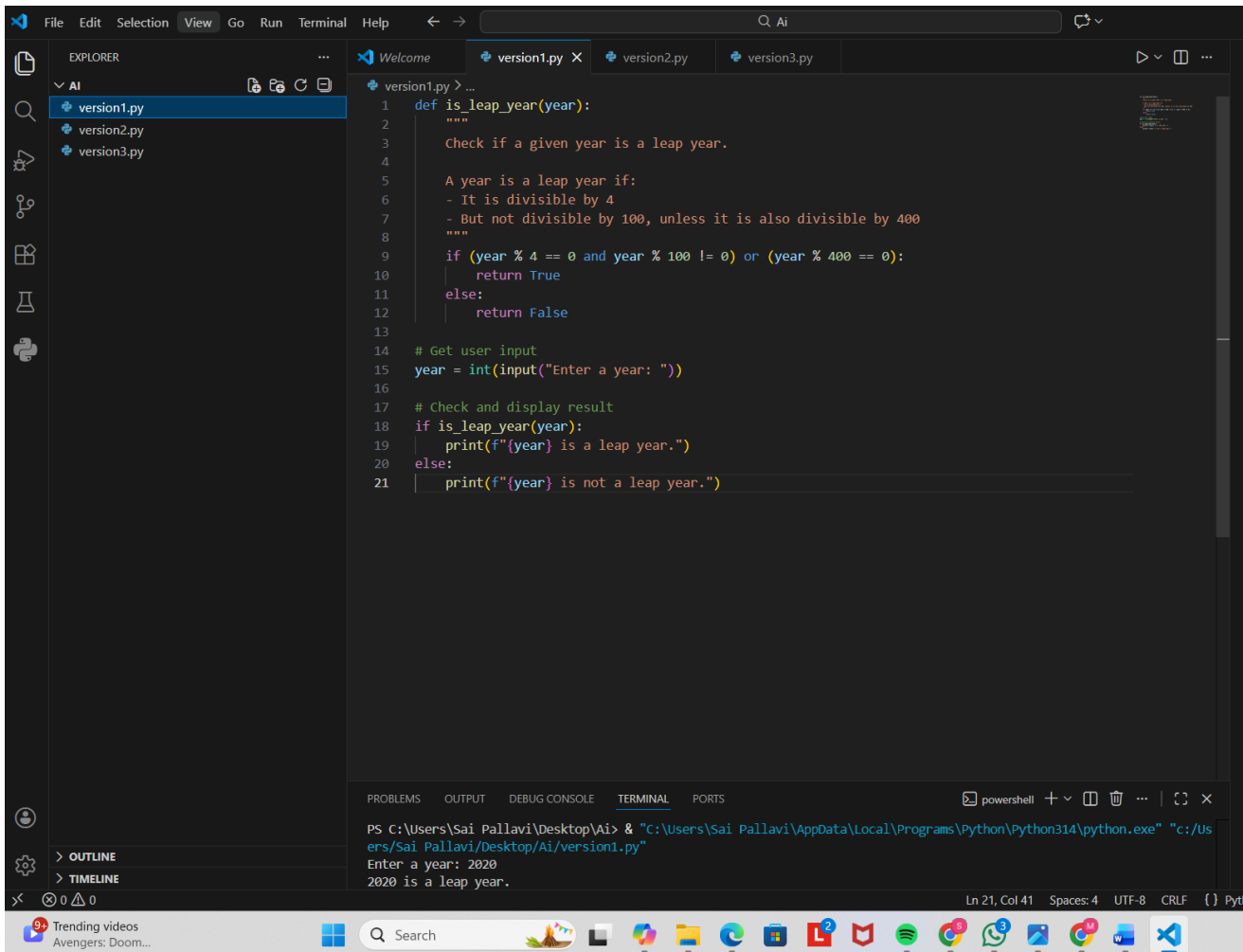
• Side-by-Side Comparison Table:

Feature	Google Gemini	GitHub Copilot
Code structure	Uses a separate function	Written directly in main logic
Readability	Very clear and modular	Simple and straightforward
Logic clarity	Easy to understand with function	Easy but less modular
Beginner friendly	High	Medium
Explanation support	Provides explanation along with code	No explanation, only code
Best use case	Learning and documentation	Fast coding inside editor

Task 3: Leap Year Validation Using Cursor AI

❖ *Scenario: You are validating a calendar module for a backend system.*

- **Prompt 1:** "Write a Python program to check whether a given year is a leap year."
- **Prompt 2:** "Write an optimized Python program with proper conditions and comments to check whether a year is a leap year."
- **Screenshot of Generated Code(V1):**

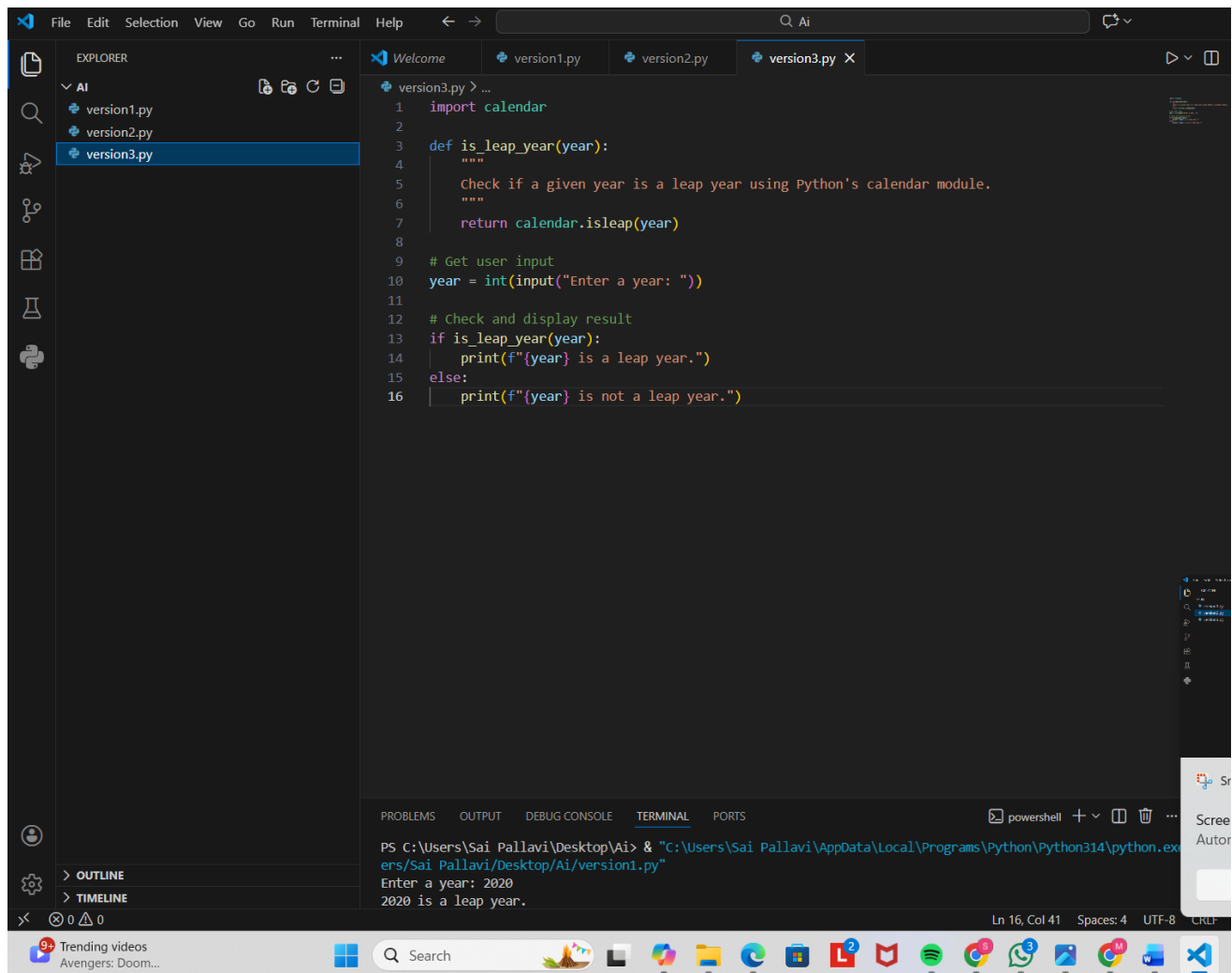


The screenshot displays the Cursor AI IDE interface. The Explorer panel on the left shows a project named 'AI' containing three files: 'version1.py', 'version2.py', and 'version3.py'. The main editor window is open to 'version1.py', which contains the following Python code:

```
1 def is_leap_year(year):
2     """
3     Check if a given year is a leap year.
4
5     A year is a leap year if:
6     - It is divisible by 4
7     - But not divisible by 100, unless it is also divisible by 400
8     """
9     if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
10        return True
11    else:
12        return False
13
14 # Get user input
15 year = int(input("Enter a year: "))
16
17 # Check and display result
18 if is_leap_year(year):
19     print(f"{year} is a leap year.")
20 else:
21     print(f"{year} is not a leap year.")
```

The bottom panel shows the TERMINAL output with the command prompt running the script. The user has entered '2020', and the output is '2020 is a leap year.' The status bar at the bottom indicates the cursor is at line 21, column 41, with 4 spaces, UTF-8 encoding, and CRLF line endings.

• Screenshot of Generated Code(V2):



The screenshot displays the Visual Studio Code interface with a Python file named `version3.py` open. The code defines a function `is_leap_year` that uses the `calendar` module to check if a year is a leap year. It prompts the user for a year and prints the result.

```
1 import calendar
2
3 def is_leap_year(year):
4     """
5     Check if a given year is a leap year using Python's calendar module.
6     """
7     return calendar.isleap(year)
8
9 # Get user input
10 year = int(input("Enter a year: "))
11
12 # Check and display result
13 if is_leap_year(year):
14     print(f"{year} is a leap year.")
15 else:
16     print(f"{year} is not a leap year.")
```

The terminal at the bottom shows the command prompt execution:

```
PS C:\Users\Sai Pallavi\Desktop\Ai> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python314\python.exe" ...
ers/Sai Pallavi/Desktop/Ai/version1.py"
Enter a year: 2020
2020 is a leap year.
```

The status bar at the bottom indicates the current position is at line 16, column 41, with 4 spaces and UTF-8 encoding.

- **Sample Input:** Enter a year: 2026
- **Sample Output:** 2026 is not a leap year.

- **Short Explanation of Logic:**

version1.py manually checks leap year rules using modulo arithmetic, making the logic explicit and educational with no dependencies. version3.py uses Python's built-in calendar.isleap(), which is shorter, cleaner, and production-ready but depends on the standard library. Both are fast and correct: the first emphasizes learning the rules, while the second emphasizes simplicity.

Task 4: Student Logic + AI Refactoring (Odd/Even Sum)

❖ *Scenario: Company policy requires developers to write logic before using AI.*

- **Prompt used:** "Refactor this Python code to improve readability and efficiency."

- **Screenshot of Generated Code**

The image displays two screenshots of a Jupyter Notebook interface, specifically the AIAC LAB 02.ipynb file. The interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help), a search bar, and a command palette. The notebook content is titled "Task 4: Student Logic + AI Refactoring (Odd/Even Sum)".

The first screenshot shows the original code, which is a function `calculate_odd_even_sums_original` that calculates the sum of odd and even numbers in a tuple. It includes a docstring, a loop to iterate through the tuple, and a return statement. Below the function, there is an example usage for the original code, showing the calculation of sums for a tuple of numbers (1, 2, 3, 4, 5, 6, 7, 8, 9, 10).

The second screenshot shows the refactored code, which is a function `calculate_odd_even_sums_refactored`. This version uses list comprehensions for a more concise and Pythonic approach. It includes a docstring, a single line for calculating the even sum, and a single line for calculating the odd sum. Below the function, there is an example usage for the refactored code, showing the calculation of sums for a tuple of numbers (11, 12, 13, 14, 15, 16, 17, 18, 19, 20). Additionally, there is an example with an empty tuple, showing that the function correctly handles edge cases.

AIAC LAB 02.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[3] ✓ Os

```
print(f" Sum of even numbers: {even_empty_ref}")
```

Original Code Results for (1, 2, 3, 4, 5, 6, 7, 8, 9, 10):
Sum of odd numbers: 25
Sum of even numbers: 30

=====

Refactored Code Results for (11, 12, 13, 14, 15, 16, 17, 18, 19, 20):
Sum of odd numbers: 75
Sum of even numbers: 80

=====

Original Code Results for ():
Sum of odd numbers: 0
Sum of even numbers: 0

Refactored Code Results for ():
Sum of odd numbers: 0
Sum of even numbers: 0

• Explanation

 write Explanation of improvements for original code and refracted code

<> Empty cell

✦ Certainly! Here's a summary of the improvements between the original and refactored code for calculating the sum of odd and even numbers:

Original Code (`calculate_odd_even_sums_original`):

- **Explicit Loop and Conditionals:** Uses a traditional `for` loop with `if-else` statements to iterate and check each number, explicitly managing `odd_sum` and `even_sum` variables.
- **Readability (Beginner):** Easy to understand for those new to programming due to its step-by-step nature.
- **Verbosity:** More lines of code are required to achieve the desired outcome.

Refactored Code (`calculate_odd_even_sums_refactored`):

- **Conciseness & Pythonic Style:** Achieves the same result in significantly fewer lines of code by using generator expressions within the built-in `sum()` function. This is considered a more idiomatic and expressive Python approach.
- **Readability (Experienced Pythonista):** For experienced Python developers, this version is often more readable as it clearly states *what* is being aggregated (sum of even numbers, sum of odd numbers) rather than *how* the iteration and conditional logic are handled.